

EXPERIMENT 1: Salary Transformation System Using Functional Programming Techniques

Aim:

To develop a Java program that increases each employee's salary by 10% using functional transformation techniques (Java Streams) and displays the updated salaries.

Algorithm:

- Step 1.** Start the program.
- Step 2.** Read the number of employees N.
- Step 3.** Read the salaries of all employees into a list.
- Step 4.** Apply a functional transformation using the Stream API.
- Step 5.** Increase each salary by 10% using the formula:
$$\text{Updated Salary} = \text{Salary} \times 1.10$$
- Step 6.** Store the transformed salaries in a new list.
- Step 7.** Display the updated salaries.
- Step 8.** Stop the program.

Java Program:

```
import java.util.*;
import java.util.stream.Collectors;

public class SalaryTransformationSystem {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        // Read number of employees
        int n = sc.nextInt();

        List<Integer> salaries = new ArrayList<>();

        // Read salaries
        for (int i = 0; i < n; i++) {
            salaries.add(sc.nextInt());
        }

        // Increase salary by 10%
        List<Integer> updatedSalaries = salaries.stream()
            .map(salary -> (int) (salary * 1.10))
            .collect(Collectors.toList());

        // Print updated salaries
        for (int salary : updatedSalaries) {
            System.out.print(salary + " ");
        }
    }
}
```

```
        sc.close();  
    }  
}
```

Sample Input:

5
1000 2000 3000 4000 5000

Sample Output:

1100 2200 3300 4400 5500

Result:

Thus, the Java program to increase employee salaries by 10% using functional transformation techniques (Java Stream API) was implemented successfully, and the updated salaries were displayed correctly.

EXPERIMENT 2: Design and Implementation of a Real-Time Stream Analytics Engine Using Stream Processing Concepts

Aim:

To develop a Java program that processes sensor temperature readings using the Java Stream API by filtering temperatures greater than 50, grouping readings by sensor ID, computing the average temperature for each sensor, and displaying the results sorted in descending order of average temperature.

Algorithm:

- Step 1.** Start the program.
- Step 2.** Read the number of sensor readings (N).
- Step 3.** Read each sensor ID and its corresponding temperature.
- Step 4.** Filter the sensor readings with temperatures greater than 50°C.
- Step 5.** Group the filtered readings based on **Sensor ID**.
- Step 6.** Calculate the average temperature for each sensor.
- Step 7.** Sort the sensors in descending order based on their average temperature.
- Step 8.** Display the Sensor ID along with its average temperature.
- Step 9.** Stop the program.

Java Program:

```
import java.util.*;
import java.util.stream.*;

class SensorReading {
    String sensorId;
    double temperature;

    SensorReading(String sensorId, double temperature) {
        this.sensorId = sensorId;
        this.temperature = temperature;
    }

    public String getSensorId() {
        return sensorId;
    }

    public double getTemperature() {
        return temperature;
    }
}

public class RealTimeStreamAnalyticsEngine {

    public static void main(String[] args) {
```

```

Scanner sc = new Scanner(System.in);

int n = sc.nextInt();

List<SensorReading> readings = new ArrayList<>();

for (int i = 0; i < n; i++) {
    String id = sc.next();
    double temp = sc.nextDouble();
    readings.add(new SensorReading(id, temp));
}

Map<String, Double> averageMap = readings.stream()
    .filter(r -> r.getTemperature() > 50)
    .collect(Collectors.groupingBy(
        SensorReading::getSensorId,
        Collectors.averagingDouble(SensorReading::getTemperature)
    ));

averageMap.entrySet().stream()
    .sorted(Map.Entry.<String, Double>comparingByValue().reversed())
    .forEach(entry ->
        System.out.println(entry.getKey() + " " +
            String.format("%.1f", entry.getValue())));
}
}

```

Sample Input:

```

6
S1 60
S2 40
S1 80
S3 70
S2 90
S3 30

```

Sample Output:

```

S2 90.0
S1 70.0
S3 70.0

```

Steps to Create and Execute the Program in Eclipse

1. Open **Eclipse IDE**.
2. Select **File** → **New** → **Java Project**.
3. Enter the project name as **RealTimeStreamAnalyticsEngine** and click **Finish**.
4. Right-click the **src** folder and select **New** → **Class**.
5. Enter the class name as **RealTimeStreamAnalyticsEngine**.

6. Select the option **public static void main(String[] args)** and click **Finish**.
7. Copy and paste the Java program into the editor.
8. Save the file (**Ctrl + S**).
9. Right-click the Java file and select **Run As → Java Application**.
10. Enter the sample input in the **Console** and press **Enter**.
11. Verify that the output displays the average temperature for each sensor after filtering, grouping, and sorting.

Result:

Thus, the Java program to implement a **Real-Time Stream Analytics Engine** using the **Java Stream API** was developed successfully. The program correctly filtered temperature readings, grouped them by sensor ID, computed the average temperature for each sensor, and displayed the results in descending order of average temperature.

EXPERIMENT 3: Design and Implementation of a Maximum Profit Analyzer Using Kadane's Algorithm

Aim:

To develop a Java program that finds the maximum possible profit obtainable from a contiguous sequence of days using **Kadane's Algorithm**.

Algorithm:

- Step 1.** Start the program.
- Step 2.** Read the number of days (N).
- Step 3.** Read the profit/loss values into an array.
- Step 4.** Initialize maxSum and currentSum with the first element.
- Step 5.** Traverse the array from the second element.
- Step 6.** Update currentSum as the maximum of:
 - a. Current element
 - b. Current element + previous currentSum
- Step 7.** Update maxSum if currentSum is greater.
- Step 8.** Print maxSum.
- Step 9.** Stop the program.

Java Program:

```
package com.example;

import java.util.Scanner;

public class MaximumProfitAnalyzer {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        int[] arr = new int[n];

        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }

        int currentSum = arr[0];
        int maxSum = arr[0];

        for (int i = 1; i < n; i++) {

            currentSum = Math.max(arr[i], currentSum + arr[i]);
            maxSum = Math.max(maxSum, currentSum);
        }
    }
}
```

```

    }

    System.out.println(maxSum);

    sc.close();
}
}

```

Sample Input:

8
-2 -3 4 -1 -2 1 5 -3

Sample Output:

7

Explanation:

The maximum profit is obtained from the contiguous subarray:

4 -1 -2 1 5

Sum = $4 + (-1) + (-2) + 1 + 5 = 7$

Hence, the maximum profit is:

7

Steps to Create and Execute the Program in Eclipse

1. Open **Eclipse IDE**.
2. Select **File** → **New** → **Java Project**.
3. Enter the project name as **MaximumProfitAnalyzer** and click **Finish**.
4. Right-click the **src** folder → **New** → **Class**.
5. Enter the class name **MaximumProfitAnalyzer**.
6. Select **public static void main(String[] args)**.
7. Copy and paste the Java program.
8. Save the file (**Ctrl + S**).
9. Right-click the Java file → **Run As** → **Java Application**.
10. Enter the sample input in the Console.
11. Verify that the output is 7.

Result:

Thus, the Java program to determine the **maximum profit from a contiguous sequence of days using Kadane's Algorithm** was implemented successfully, and the maximum subarray sum was obtained correctly.

EXPERIMENT 4: Design and Implementation of an Intelligent DNA Pattern Search Using KMP Algorithm

Aim:

To develop a Java program that identifies all occurrences of a DNA pattern within a DNA sequence using the **Knuth-Morris-Pratt (KMP) Algorithm**.

Algorithm:

Step 1: Build the LPS (Longest Prefix Suffix) Array

1. Initialize the LPS array.
2. Compare characters in the pattern.
3. Store the length of the longest proper prefix which is also a suffix.

Step 2: Search the Pattern

1. Read the DNA sequence (**T**) and pattern (**P**).
2. Compare characters of the text and pattern.
3. If characters match, move both pointers.
4. If the complete pattern matches, print the starting index.
5. If mismatch occurs, use the LPS array to skip unnecessary comparisons.
6. Continue until the end of the DNA sequence.

Java Program:

```
package com.example;

import java.util.*;

public class IntelligentDNAPatternSearch {

    // Function to build LPS array
    static int[] computeLPS(String pattern) {

        int m = pattern.length();
        int[] lps = new int[m];

        int len = 0;
        int i = 1;

        while (i < m) {

            if (pattern.charAt(i) == pattern.charAt(len)) {
                len++;
                lps[i] = len;
                i++;
            } else {

                if (len != 0) {
```

```

        len = lps[len - 1];
    } else {
        lps[i] = 0;
        i++;
    }
}
}

return lps;
}

// KMP Search
static void KMPSearch(String text, String pattern) {

    int[] lps = computeLPS(pattern);

    int i = 0;
    int j = 0;

    while (i < text.length()) {

        if (text.charAt(i) == pattern.charAt(j)) {
            i++;
            j++;
        }

        if (j == pattern.length()) {
            System.out.print((i - j) + " ");
            j = lps[j - 1];
        }

        else if (i < text.length() &&
            text.charAt(i) != pattern.charAt(j)) {

            if (j != 0)
                j = lps[j - 1];
            else
                i++;
        }
    }
}

public static void main(String[] args) {

    Scanner sc = new Scanner(System.in);

    String text = sc.nextLine();

```

```

        String pattern = sc.nextLine();

        KMPSearch(text, pattern);

        sc.close();
    }
}

```

Sample Input:

AABAACAADAABAABAAABA

Sample Output:

0 9 12

Explanation:

DNA Sequence:

AABAACAADAABAABA

Pattern:

AABA

The pattern is found at:

- Index 0
- Index 9
- Index 12

Hence the output is:

0 9 12

Steps to Create and Execute the Program in Eclipse

1. Open **Eclipse IDE**.
2. Create a new **Java Project**.
3. Right-click the **src** folder → **New** → **Class**.
4. Name the class **IntelligentDNAPatternSearch**.
5. Copy and paste the above program.
6. Save the file.
7. Right-click the file → **Run As** → **Java Application**.
8. Enter the DNA sequence followed by the pattern.
9. Verify that the starting indices of all occurrences are displayed.

Result::

Thus, the Java program to identify occurrences of a DNA pattern using the **Knuth-Morris-Pratt (KMP) Algorithm** was implemented successfully, and all starting indices of the pattern in the DNA sequence were displayed correctly.

EXPERIMENT 5: Design and Implementation of a Banking Transaction System Using Object-Oriented Programming

Aim:

To develop a Java program that implements a **Banking Transaction System** using Object-Oriented Programming (OOP) concepts to perform **deposit** and **withdrawal** operations and display the final account balance.

Algorithm:

- Step 1. Start the program.
- Step 2. Create a **BankAccount** class with a balance variable.
- Step 3. Define methods:
 - i. deposit(amount) to add money to the balance.
 - ii. withdraw(amount) to subtract money from the balance.
- Step 4. Read the number of transactions (**N**).
- Step 5. For each transaction:
 - i. Read the operation (**Deposit** or **Withdraw**) and the amount.
 - ii. Perform the corresponding operation.
- Step 6. Display the final balance.
- Step 7. Stop the program.

Java Program:

```
package com.example;

import java.util.Scanner;

class BankAccount {

    int balance = 0;

    void deposit(int amount) {
        balance = balance + amount;
    }

    void withdraw(int amount) {
        balance = balance - amount;
    }

    void displayBalance() {
        System.out.println(balance);
    }
}
```

```

public class BankingTransactionSystem {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        BankAccount account = new BankAccount();

        int n = sc.nextInt();

        for (int i = 0; i < n; i++) {

            String operation = sc.next();
            int amount = sc.nextInt();

            if (operation.equalsIgnoreCase("Deposit")) {
                account.deposit(amount);
            }
            else if (operation.equalsIgnoreCase("Withdraw")) {
                account.withdraw(amount);
            }
        }

        account.displayBalance();

        sc.close();
    }
}

```

Sample Input:

3
 Deposit 5000
 Withdraw 2000
 Deposit 1000

Sample Output:

4000

Explanation:

Initial Balance = 0

- Deposit **5000** → Balance = **5000**
- Withdraw **2000** → Balance = **3000**
- Deposit **1000** → Balance = **4000**

Final Balance:

4000

Steps to Create and Execute the Program in Eclipse

1. Open **Eclipse IDE**.
2. Create a **New Java Project** named **BankingTransactionSystem**.
3. Right-click the **src** folder → **New** → **Class**.
4. Enter the class name **BankingTransactionSystem**.
5. Copy and paste the Java program.
6. Save the file (**Ctrl + S**).
7. Right-click the Java file → **Run As** → **Java Application**.
8. Enter the sample input in the **Console**.
9. Verify that the final balance is displayed correctly.

Result:

Thus, the Java program to implement a **Banking Transaction System** using **Object-Oriented Programming** was developed successfully. The program correctly performed **deposit** and **withdrawal** operations and displayed the final account balance.

EXPERIMENT 6: Design and Implementation of a Ride Sharing Platform Simulator Using Object-Oriented Programming

Aim:

To develop a Java program that simulates a **Ride Sharing Platform** using Object-Oriented Programming concepts such as **inheritance**, **polymorphism**, and **exception handling** to calculate fares for **Bike**, **Auto**, and **Cab** rides.

Algorithm:

- Step 1. Start the program.
- Step 2. Create a base class **Vehicle** with a method calculateFare().
- Step 3. Create subclasses **Bike**, **Auto**, and **Cab** that override the calculateFare() method.
- Step 4. Read the number of rides (**N**).
- Step 5. For each ride:
 - i. Read the ride type and distance.
 - ii. Create the corresponding object.
 - iii. If the ride type is invalid, throw an exception.
 - iv. Otherwise, calculate and display the fare.
- Step 6. Stop the program.

Java Program:

```
package com.example;

import java.util.Scanner;

// Base class
class Vehicle {
    public int calculateFare(int distance) {
        return 0;
    }
}

// Bike class
class Bike extends Vehicle {
    public int calculateFare(int distance) {
        return distance * 5;
    }
}

// Auto class
class Auto extends Vehicle {
    public int calculateFare(int distance) {
        return distance * 12;
    }
}
```

```

// Cab class
class Cab extends Vehicle {
    public int calculateFare(int distance) {
        return distance * 12;
    }
}

public class RideSharingPlatformSimulator {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        for (int i = 0; i < n; i++) {

            String type = sc.next();
            int distance = sc.nextInt();

            Vehicle vehicle = null;

            try {

                if (type.equalsIgnoreCase("Bike")) {
                    vehicle = new Bike();
                }
                else if (type.equalsIgnoreCase("Auto")) {
                    vehicle = new Auto();
                }
                else if (type.equalsIgnoreCase("Cab")) {
                    vehicle = new Cab();
                }
                else {
                    throw new IllegalArgumentException("Invalid Ride");
                }

                System.out.println(vehicle.calculateFare(distance));

            } catch (IllegalArgumentException e) {
                System.out.println(e.getMessage());
            }
        }

        sc.close();
    }
}

```

Sample Input:

3
Bike 10
Cab 15
Auto 8

Sample Output:

50
180
96

Explanation:

Ride Type	Distance	Rate (₹/km)	Fare
Bike	10 km	₹5	₹50
Cab	15 km	₹12	₹180
Auto	8 km	₹12	₹96

Object-Oriented Concepts Used

- **Inheritance:** Bike, Auto, and Cab inherit from the Vehicle class.
- **Polymorphism:** Each subclass overrides the calculateFare() method.
- **Exception Handling:** Handles invalid ride types using IllegalArgumentException.

Steps to Create and Execute the Program in Eclipse

1. Open **Eclipse IDE**.
2. Create a **New Java Project** named **RideSharingPlatformSimulator**.
3. Right-click the **src** folder → **New** → **Class**.
4. Enter the class name **RideSharingPlatformSimulator**.
5. Copy and paste the Java program into the editor.
6. Save the file (**Ctrl + S**).
7. Right-click the Java file → **Run As** → **Java Application**.
8. Enter the sample input in the **Console**.
9. Verify that the calculated fare for each ride is displayed correctly.

Result:

Thus, the Java program to simulate a **Ride Sharing Platform** using **Object-Oriented Programming concepts** was implemented successfully. The program correctly calculated fares for **Bike**, **Auto**, and

Cab rides using **inheritance** and **polymorphism**, and handled invalid ride types through **exception handling**.

EXPERIMENT 7: Design and Implementation of a Hashtag Frequency Counter Using Maps

Aim:

To develop a Java program that counts the frequency of hashtags appearing in social media posts using the **HashMap** collection.

Algorithm:

- Step 1. Start the program.
- Step 2. Read the number of hashtags (**N**).
- Step 3. Create a **HashMap** to store hashtags and their frequencies.
- Step 4. Read each hashtag.
- Step 5. If the hashtag already exists in the HashMap, increment its count.
- Step 6. Otherwise, insert the hashtag with a count of **1**.
- Step 7. Display each hashtag along with its frequency.
- Step 8. Stop the program.

Java Program:

```
package com.example;

import java.util.HashMap;
import java.util.Map;
import java.util.Scanner;

public class HashtagFrequencyCounter {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        HashMap<String, Integer> hashtagCount = new HashMap<>();

        for (int i = 0; i < n; i++) {

            String hashtag = sc.next();

            if (hashtagCount.containsKey(hashtag)) {
                hashtagCount.put(hashtag, hashtagCount.get(hashtag) + 1);
            } else {
                hashtagCount.put(hashtag, 1);
            }
        }
    }
}
```

```

        for (Map.Entry<String, Integer> entry : hashtagCount.entrySet()) {
            System.out.println(entry.getKey() + " " + entry.getValue());
        }

        sc.close();
    }
}

```

Sample Input:

```

5
java
python
java
ai
python

```

Sample Output:

```

java 2
python 2
ai 1

```

Note: Since **HashMap** does not maintain insertion order, the output order may vary. For example:

```

python 2
java 2
ai 1

```

If you want the output to appear in the **same order as the input**, use **LinkedHashMap** instead of **HashMap**.

Explanation:

Input hashtags:

```

java
python
java
ai
python

```

Frequency count:

Hashtag	Frequency
java	2
python	2
ai	1

Steps to Create and Execute the Program in Eclipse

1. Open **Eclipse IDE**.
2. Create a **New Java Project** named **HashtagFrequencyCounter**.
3. Right-click the **src** folder and select **New → Class**.
4. Enter the class name **HashtagFrequencyCounter**.
5. Copy and paste the Java program into the editor.
6. Save the file (**Ctrl + S**).
7. Right-click the Java file and select **Run As → Java Application**.
8. Enter the sample input in the **Console**.
9. Verify that the hashtag frequencies are displayed correctly.

Result

Thus, the Java program to **count the frequency of hashtags using a HashMap** was implemented

successfully. The program correctly counted and displayed the frequency of each hashtag entered by the user.

EXPERIMENT 8: Design and Implementation of a Smart Traffic Navigation System Using Graph Representation

Aim:

To develop a Java program that determines whether a route exists between two city junctions using **graph representation with collections** and **Breadth-First Search (BFS)**.

Algorithm

- Step 1.** Start the program.
- Step 2.** Read the number of junctions (**N**) and roads (**M**).
- Step 3.** Create an adjacency list using a **HashMap**.
- Step 4.** Read the connected junction pairs and build the graph.
- Step 5.** Read the source and destination junctions.
- Step 6.** Perform **Breadth-First Search (BFS)** starting from the source.
- Step 7.** If the destination is reached, print **YES**.
- Step 8.** Otherwise, print **NO**.
- Step 9.** Stop the program.

Java Program:

```
package com.example;

import java.util.*;

public class SmartTrafficNavigationSystem {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int N = sc.nextInt();
        int M = sc.nextInt();

        // Graph using adjacency list
        HashMap<Integer, ArrayList<Integer>> graph = new HashMap<>();

        for (int i = 1; i <= N; i++) {
            graph.put(i, new ArrayList<>());
        }

        // Read roads
        for (int i = 0; i < M; i++) {

            int u = sc.nextInt();
            int v = sc.nextInt();
```

```

        graph.get(u).add(v);
        graph.get(v).add(u); // Undirected graph
    }

    int source = sc.nextInt();
    int destination = sc.nextInt();

    Queue<Integer> queue = new LinkedList<>();
    boolean[] visited = new boolean[N + 1];

    queue.add(source);
    visited[source] = true;

    boolean found = false;

    while (!queue.isEmpty()) {

        int current = queue.poll();

        if (current == destination) {
            found = true;
            break;
        }

        for (int neighbor : graph.get(current)) {

            if (!visited[neighbor]) {
                visited[neighbor] = true;
                queue.add(neighbor);
            }
        }
    }

    if (found)
        System.out.println("YES");
    else
        System.out.println("NO");

    sc.close();
}
}

```

Sample Input:

```

5 4
1 2
2 3

```

3 4
4 5
1 5

Sample Output:

YES

Explanation:

Graph:

1 ---- 2 ---- 3 ---- 4 ---- 5

Source = 1

Destination = 5

A path exists:

1 → 2 → 3 → 4 → 5

Hence, the output is:

YES

Steps to Create and Execute the Program in Eclipse

1. Open **Eclipse IDE**.
2. Create a **New Java Project** named **SmartTrafficNavigationSystem**.
3. Right-click the **src** folder and select **New → Class**.
4. Enter the class name **SmartTrafficNavigationSystem**.
5. Copy and paste the Java program into the editor.
6. Save the file (**Ctrl + S**).
7. Right-click the Java file and select **Run As → Java Application**.
8. Enter the sample input in the **Console**.
9. Verify that the program displays **YES** if a route exists; otherwise, it displays **NO**.

Result

Thus, the Java program to implement a **Smart Traffic Navigation System** using **graph representation**

with **collections** and **Breadth-First Search (BFS)** was implemented successfully. The program correctly determined whether a route exists between the specified source and destination junctions.

EXPERIMENT 9: Design and Implementation of Calculator Unit Testing Using Junit

Aim:

To develop a Java program that performs **addition** and **division** operations and verifies their correctness using **JUnit unit testing**.

Algorithm:

- Step 1.** Start the program.
- Step 2.** Create a Calculator class with methods:
 - i. add(a, b)
 - ii. divide(a, b)
- Step 3.** Create a JUnit test class.
- Step 4.** Write a test case for the addition operation.
- Step 5.** Write a test case for the division operation.
- Step 6.** Compare the expected and actual results using JUnit assertions.
- Step 7.** Display **Test Passed** if all test cases are successful.
- Step 8.** Stop the program.

Calculator.java:

```
package com.example;

public class Calculator {

    public int add(int a, int b) {
        return a + b;
    }

    public int divide(int a, int b) {
        return a / b;
    }
}
```

CalculatorTest.java (JUnit 5):

```
package com.example;

import static org.junit.jupiter.api.Assertions.assertEquals;
import org.junit.jupiter.api.Test;

public class CalculatorTest {
    Calculator cal = new Calculator();

    @Test
```

```

void testAddition() {
    assertEquals(30, cal.add(10, 20));
}

@Test
void testDivision() {
    assertEquals(5, cal.divide(10, 2));
}
}

```

Sample Input:

10 + 20

Expected Test Result:

Test Passed

Explanation:

- Addition Test:
 - Expected = **30**
 - Actual = **30**
 - **Passed**
- Division Test:
 - Expected = **5**
 - Actual = **5**
 - **Passed**

Since both test cases pass, JUnit reports the tests as successful.

Steps to Create and Execute the Program in Eclipse

1. Open **Eclipse IDE**.
2. Create a **New Java Project** named **CalculatorUnitTesting**.
3. Create a package (e.g., com.example).
4. Create a class named **Calculator** and add the calculator methods.
5. Add **JUnit 5** to the project:
 - Right-click the project → **Build Path** → **Add Libraries...**
 - Select **JUnit** → **JUnit 5** → **Finish**.
6. Create a new **JUnit Test Case** named **CalculatorTest**.
7. Copy and paste the JUnit test code.
8. Save the files.
9. Right-click **CalculatorTest.java** → **Run As** → **JUnit Test**.
10. Verify that all test cases pass (green bar in Eclipse).

Result:

Thus, the Java program for **Calculator Unit Testing** was implemented successfully using **JUnit 5**. The addition and division operations were tested, and all test cases passed successfully.

EXPERIMENT 10: Design and Implementation of Secure Authentication Stress Testing Using Automated Test Cases

Aim:

To develop a Java program that validates login credentials and performs secure authentication testing by checking username and password constraints, displaying **SUCCESS** or **FAILURE** for each login attempt.

Algorithm:

- Step 1.** Start the program.
- Step 2.** Read the number of login attempts (**N**).
- Step 3.** For each login attempt:
 - i. Read username and password.
 - ii. Check whether:
 - 1. Username length is between **3 and 20** characters.
 - 2. Password length is between **6 and 20** characters.
 - iii. If both conditions are satisfied:
 - 1. Print **SUCCESS**.
 - iv. Otherwise:
 - 1. Print **FAILURE**.
- Step 4.** Repeat until all login attempts are processed.
- Step 5.** Stop the program.

Java Program:

```
package com.example;

import java.util.Scanner;

public class SecureAuthenticationStressTesting {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        for (int i = 0; i < n; i++) {
            String username = sc.next();
            String password = sc.next();
            if (username.length() >= 3 && username.length() <= 20 &&
                password.length() >= 6 && password.length() <= 20) {

                System.out.println("SUCCESS");
            } else {
                System.out.println("FAILURE");
            }
        }
        sc.close();
    }
}
```

```
}}
```

Sample Input:

3

admin admin123

ab 123

user pass

Sample Output:

SUCCESS

FAILURE

FAILURE

Explanation:

Username	Password	Validation Result
admin	admin123	SUCCESS
ab	123	FAILURE (username < 3 and password < 6)
user	pass	FAILURE (password < 6)

Testing Scenarios Covered;

Test Type	Example
Boundary Value Testing	Username length = 3, Password length = 6
Exception Testing	Null/empty input, invalid format
Timeout Testing	Large number of login attempts (up to 10^5)
Parameterized Testing	Multiple username-password combinations
Edge Case Validation	Minimum and maximum allowed lengths

Steps to Create and Execute the Program in Eclipse

1. Open **Eclipse IDE**.
2. Create a **New Java Project** named **SecureAuthenticationStressTesting**.
3. Right-click the **src** folder → **New** → **Class**.
4. Enter the class name **SecureAuthenticationStressTesting**.
5. Copy and paste the Java program.
6. Save the file (**Ctrl + S**).
7. Right-click the Java file → **Run As** → **Java Application**.
8. Enter the sample input in the Console.
9. Verify that the program displays **SUCCESS** or **FAILURE** for each login attempt.

Result:

Thus, the Java program for **Secure Authentication Stress Testing** was implemented successfully. The program validated username and password constraints efficiently and correctly displayed **SUCCESS** or **FAILURE** for each login attempt.